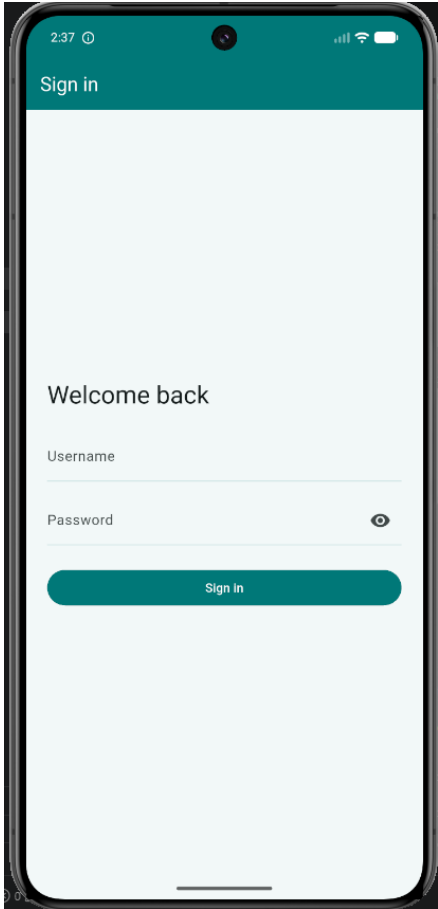


Lab Activity 4

Submission Form

Name	Jamaine Grace Tuazon
Section	INF231
Date	August 21, 2026

Place output based on the source code.





Place your sample code and output based on the enhancement. You may add rows or cells if needed.

Source code – [name of the file]

cart_screen.dart

```
42 //
43
44 try {
45   // Read the saved user first, then use their id to get their cart.
46   final user = await _userService.getUser();
47   final carts = await _cartService.getCartsByUserId(user.id);
48   if (!mounted) return;
49   setState(() {
50     // takes the first cart
51     _cart = carts.isNotEmpty ? carts.first : null;
52     _isLoading = false;
53   });
54 } catch (e) {
55   if (!mounted) return;
56   setState(() {
57     _isLoading = false;
58     _errorMessage = e.toString();
59   });
60 }
61
62 // Adds up the quantity of every item
```

home_screen.dart

```
    ],
    ), // AppBar
    body: PageView(
      physics: const NeverScrollableScrollPhysics(),
      controller: _pageController,
      children: const <Widget>[
        ProductScreen(),
        CartScreen(),
        ProfileScreen(),
      ], // <Widget>[]
      onPageChanged: (page) {
        setState(() {
          _selectedIndex = page;
        });
      },
    ), // PageView
```

splash_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';

import '../constants.dart';
import '../widgets/custom_text.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    // Start the timer
    _goToHome();
  }

  //automatic timer for splash screen then proceed to homescreen
  Future<void> _goToHome() async {
    // Change the number here if you want a shorter or longer splash
    await Future<void>.delayed(const Duration(seconds: 5));

    if (!mounted) return;
    Navigator.pushReplacementNamed(context, '/home');
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;

    //splash screens structure
    return Scaffold(
      backgroundColor: theme.scaffoldBackgroundColor,
      body: SafeArea(
        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Image.asset(
                'assets/images/nubdexchange_logo.png',
                width: 200.w,
              ),
              SizedBox(height: 24.h),
              CustomText(
                text: 'NU Online Shopping Center',
                fontSize: 20.sp,
                fontWeight: FontWeight.bold,
                textAlign: TextAlign.center,
                color: isDark ? AppColors.sand : AppColors.tealDeep,
              ),
              SizedBox(height: 8.h),
              CustomText(
                text: 'Your campus online store',
                fontSize: 13.sp,
                textAlign: TextAlign.center,
                color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



profile_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import '../constants.dart';
import '../models/user.dart';
import '../services/user_service.dart';
import '../widgets/custom_text.dart';

// This screen shows the data of the user who signed in using UserService
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  // loads the saved user data
  final UserService _userService = UserService();

  User? _user;
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    // loads the saved user right
    _loadUser();
  }

  // Gets the saved user and turns it into a User model
  Future<void> _loadUser() async {
    setState(() {
      _isLoading = true;
    });

    final user = await _userService.getUser();
    if (!mounted) return;
    setState(() {
      _user = user;
      _isLoading = false;
    });
  }

  // logout function to erases all the saved user data on the phone
  Future<void> _logout() async {
    await _userService.logout();
    if (!mounted) return;
    //navigated back to signin screen
    Navigator.pushReplacementNamed(context, '/signin');
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;

    // Still reading the saved data
    if (_isLoading) {
      return const Center(child: CircularProgressIndicator());
    }

    final user = _user;

    if (user == null) {
      return Center(
        child: CustomText(
          text: 'No user data found',
          fontSize: 14.sp,
          fontWeight: FontWeight.bold,
        ),
      );
    }

    //layout/structure of the profile screen
    return SingleChildScrollView(
      child: Column(
        children: [
          _buildHeader(user, theme, isDark),
          SizedBox(height: 8.h),
          CustomText(
            text: user.displayName,
            fontSize: 22.sp,
            fontWeight: FontWeight.bold,
            textAlign: TextAlign.center,
          ),
          SizedBox(height: 2.h),
          CustomText(
            text: '${user.userName}',
            fontSize: 13.sp,
            textAlign: TextAlign.center,
            color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
          ),
          SizedBox(height: 24.h),
          Padding(
            padding: EdgeInsets.symmetric(horizontal: 16.w),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                _buildSectionTitle('Account Information', isDark),
                SizedBox(height: 8.h),
                _buildDetailCard(
                  theme: theme,
                  isDark: isDark,
                  children: [
                    _buildDetailTile(
                      icon: Icons.person,
                      label: 'First name',
                      value: user.firstName,
                      theme: theme,
                      isDark: isDark,
                    ),
                    _buildDivider(isDark),
                    _buildDetailTile(
                      icon: Icons.person_outline,
                      label: 'Last name',
                      value: user.lastName,
                      theme: theme,
                      isDark: isDark,
                    ),
                    _buildDivider(isDark),
                    _buildDetailTile(
                      icon: Icons.tag,
                      label: 'Username',
                      value: user.userName,
                      theme: theme,
                      isDark: isDark,
                    ),
                    _buildDivider(isDark),
                    _buildDetailTile(
                      icon: Icons.alternate_email,
                      label: 'Email',
                      value: user.email,
                      theme: theme,
                      isDark: isDark,
                    ),
                  ],
                ),
                SizedBox(height: 24.h),
                SizedBox(
                  width: double.infinity,
                  height: 48.h,
                  child: OutlinedButton.icon(
                    onPressed: _logout,
                    icon: Icon(
                      Icons.logout,
                      size: 28.sp,
                      color: theme.colorScheme.error,
                    ),
                    label: CustomText(
                      text: 'Log out',
                      fontSize: 15.sp,
                      fontWeight: FontWeight.bold,
                      color: theme.colorScheme.error,
                    ),
                    style: OutlinedButton.styleFrom(
                      side: BorderSide(color: theme.colorScheme.error),
                      shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(12.r),
                      ),
                    ),
                  ),
                ),
                SizedBox(height: 24.h),
              ],
            ),
          ),
        ],
      );
    }
  }
}
```

```
//header, contains the gradient banner and profile picture
Widget _buildHeader(User user, ThemeData theme, bool isDark) {
  return SizedBox(
    height: 66.h + 184.r,
    child: Stack(
      alignment: Alignment.topCenter,
      children: [
        Positioned(
          top: 0,
          left: 0,
          right: 0,
          child: Container(
            height: 120.h,
            decoration: BoxDecoration(
              gradient: LinearGradient(
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
                colors: isDark
                  ? [AppColors.tealDeep, AppColors.darkCard]
                  : [AppColors.teal, AppColors.tealLight],
              ),
              borderRadius: BorderRadius.only(
                bottomLeft: Radius.circular(24.r),
                bottomRight: Radius.circular(24.r),
              ),
            ),
          ),
        ),
        Positioned(
          top: 66.h,
          child: Container(
            padding: EdgeInsets.all(4.r),
            decoration: BoxDecoration(
              color: theme.scaffoldBackgroundColor,
              shape: BoxShape.circle,
            ),
            child: CircleAvatar(
              radius: 48.r,
              backgroundColor:
                isDark ? AppColors.darkCard : AppColors.lightCard,
              backgroundImage:
                user.image.isNotEmpty ? NetworkImage(user.image) : null,
              child: user.image.isEmpty
                ? Icon(
                    Icons.person,
                    size: 48.sp,
                    color: isDark
                      ? AppColors.darkMuted
                      : AppColors.lightMuted,
                )
                : null,
            ),
          ),
        ),
      ],
    );
  }

  Widget _buildSectionTitle(String title, bool isDark) {
    return CustomText(
      text: title.toUpperCase(),
      fontSize: 11.sp,
      fontWeight: FontWeight.bold,
      letterSpacing: 1.0,
      color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
    );
  }

  Widget _buildDetailCard({
    required ThemeData theme,
    required bool isDark,
    required List<Widget> children,
  }) {
    return Card(
      elevation: 0,
      margin: EdgeInsets.zero,
      color: theme.cardTheme.color,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16.r),
        side: BorderSide(
          color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
        ),
      ),
      child: Padding(
        padding: EdgeInsets.symmetric(horizontal: 14.w, vertical: 6.h),
        child: Column(children: children),
      ),
    );
  }

  //for labels
  Widget _buildDetailTile({
    required IconData icon,
    required String label,
    required String value,
    required ThemeData theme,
    required bool isDark,
  }) {
    return Padding(
      padding: EdgeInsets.symmetric(vertical: 10.h),
      child: Row(
        children: [
          Container(
            width: 36.w,
            height: 36.h,
            decoration: BoxDecoration(
              color: theme.colorScheme.primaryContainer,
              borderRadius: BorderRadius.circular(10.r),
            ),
            child: Icon(
              icon,
              size: 18.sp,
              color: theme.colorScheme.onPrimaryContainer,
            ),
          ),
          SizedBox(width: 12.w),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                CustomText(
                  text: label,
                  fontSize: 11.sp,
                  color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
                ),
                SizedBox(height: 2.h),
                CustomText(
                  text: value.isEmpty ? '-' : value,
                  fontSize: 13.sp,
                  fontWeight: FontWeight.w600,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                ),
              ],
            ),
          ),
        ],
      );
  }

  // row divider
  Widget _buildDivider(bool isDark) {
    return Divider(
      height: 1,
      thickness: 1,
      color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
    );
  }
}
```



signin_screeb.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import '../constants.dart';
import '../services/user_service.dart';
import '../widgets/custom_text.dart';
class SignInScreen extends StatefulWidget {
  const SignInScreen({super.key});

  @override
  State<SignInScreen> createState() => _SignInScreenState();
}

class _SignInScreenState extends State<SignInScreen> {
  // restrictions, need to username and pass to be filled up
  final _formKey = GlobalKey<FormState>();
  // Controllers
  final _usernameController = TextEditingController();
  final _passwordController = TextEditingController();
  // login API
  final _userService = UserService();
  bool _isLoading = false;
  bool _obscurePassword = true;
  bool _isCheckingLogin = true;

  @override
  void initState() {
    super.initState();
    // Checks if the device already has a saved login
    _checkExistingLogin();
  }

  // maintains sign in state of user if hindi mag log out
  Future<void> _checkExistingLogin() async {
    bool isLoggedIn = false;
    try {
      isLoggedIn = await _userService.isLoggedIn();
    } catch (e) {
      debugPrint('TOKEN CHECK FAILED: $e');
      isLoggedIn = false;
    }

    if (!mounted) return;

    if (isLoggedIn) {
      Navigator.pushReplacementNamed(context, '/splash');
    } else {
      setState(() => _isCheckingLogin = false);
    }
  }

  @override
  void dispose() {
    // Clean up the controllers
    _usernameController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  // Sign in function
  Future<void> _login() async {
    if (!(_formKey.currentState?.validate() ?? false)) return;

    setState(() => _isLoading = true);
    try {
      await _userService.loginUser(
        _usernameController.text.trim(),
        _passwordController.text.trim(),
      );
      if (!mounted) return;
      //splash screen triggers
      Navigator.pushReplacementNamed(context, '/splash');
    } catch (e) {
      debugPrint('SIGN IN FAILED: $e');
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: CustomText(
            text: 'Unable to sign in.\n$e',
            fontSize: 12.sp,
            fontWeight: FontWeight.w500,
          ),
          duration: const Duration(seconds: 8),
        ),
      );
    } finally {
      // Hide the spinner whether the login worked or not
      if (mounted) setState(() => _isLoading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;

    if (_isCheckingLogin) {
      return Scaffold(
        backgroundColor: theme.scaffoldBackgroundColor,
        body: const Center(child: CircularProgressIndicator()),
      );
    }
  }
}
```

```
//sign in screen UI structure
return Scaffold(
  backgroundColor: theme.scaffoldBackgroundColor,
  body: SafeArea(
    child: Center(
      child: SingleChildScrollView(
        padding: EdgeInsets.all(24.r),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Image.asset(
              'assets/images/nubdexchange_logo.png',
              width: 160.w,
            ),
            SizedBox(height: 16.h),
            CustomText(
              text: 'NU Online Shopping Center',
              fontSize: 18.sp,
              fontWeight: FontWeight.bold,
              textAlign: TextAlign.center,
              color: isDark ? AppColors.sand : AppColors.tealDeep,
            ),
            SizedBox(height: 6.h),
            CustomText(
              text: 'Sign in to continue shopping',
              fontSize: 13.sp,
              textAlign: TextAlign.center,
              color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
            ),
            SizedBox(height: 24.h),
            Card(
              elevation: 0,
              color: theme.cardTheme.color,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(16.r),
                side: BorderSide(
                  color:
                    isDark ? AppColors.darkBorder : AppColors.lightBorder,
                ),
            ),
            child: Padding(
              padding: EdgeInsets.all(20.r),
              child: Form(
                key: _formKey,
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    TextFormField(
                      controller: _usernameController,
                      textInputAction: TextInputAction.next,
                      style: TextStyle(
                        fontFamily: 'Poppins',
                        fontSize: 14.sp,
                      ),
                      decoration: _buildFieldDecoration(
                        label: 'Username',
                        icon: Icons.person_outline,
                        theme: theme,
                        isDark: isDark,
                      ),
                      // Shows a red message if the field is empty
                      validator: (value) =>
                        value == null || value.trim().isEmpty
                          ? 'Enter your username'
                          : null,
                    ),
                    SizedBox(height: 16.h),
                    TextFormField(
                      controller: _passwordController,
                      obscureText: _obscurePassword,
                      onFieldSubmitted: (_) => _login(),
                      style: TextStyle(
                        fontFamily: 'Poppins',
                        fontSize: 14.sp,
                      ),
                      decoration: _buildFieldDecoration(
                        label: 'Password',
                        icon: Icons.lock_outline,
                        theme: theme,
                        isDark: isDark,
                      ),
                      suffix: IconButton(
                        tooltip: _obscurePassword
                          ? 'Show password'
                          : 'Hide password',
                        icon: Icon(
                          _obscurePassword
                            ? Icons.visibility
                            : Icons.visibility_off,
                          size: 20.sp,
                          color: isDark
                            ? AppColors.darkMuted
                            : AppColors.lightMuted,
                        ),
                      onPressed: () => setState(
                        () => _obscurePassword = !_obscurePassword,
                      ),
                    ),
                    validator: (value) => value == null || value.isEmpty
                      ? 'Enter your password'
                      : null,
                    ),
                    SizedBox(height: 24.h),
                    SizedBox(
                      height: 48.h,
                      child: ElevatedButton(
                        // Turned off while loading so the user cannot
                        // press it twice
                        onPressed: _isLoading ? null : _login,
                        style: ElevatedButton.styleFrom(
                          shape: RoundedRectangleBorder(
                            borderRadius: BorderRadius.circular(12.r),
                          ),
                        ),
                        child: CustomText(
                          text: 'Sign in',
                          fontSize: 15.sp,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ],
        ),
      ),
    ),
  );

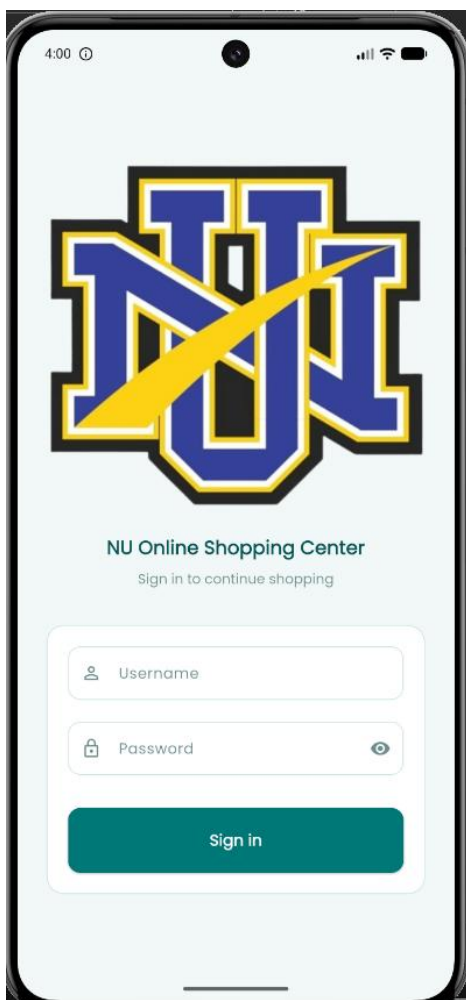
//username and pass field design
InputDecoration _buildFieldDecoration({
  required String label,
  required IconData icon,
  required ThemeData theme,
  required bool isDark,
  Widget? suffix,
}) {
  return InputDecoration(
    labelText: label,
    labelStyle: TextStyle(
      fontFamily: 'Poppins',
      fontSize: 14.sp,
      color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
    ),
    prefixIcon: Icon(
      icon,
      size: 20.sp,
      color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
    ),
    suffixIcon: suffix,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12.r),
      borderSide: BorderSide(
        color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
      ),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12.r),
      borderSide: BorderSide(
        color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
      ),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12.r),
      borderSide: BorderSide(color: theme.colorScheme.primary),
    ),
  );
}
```

Output – [name of the output]

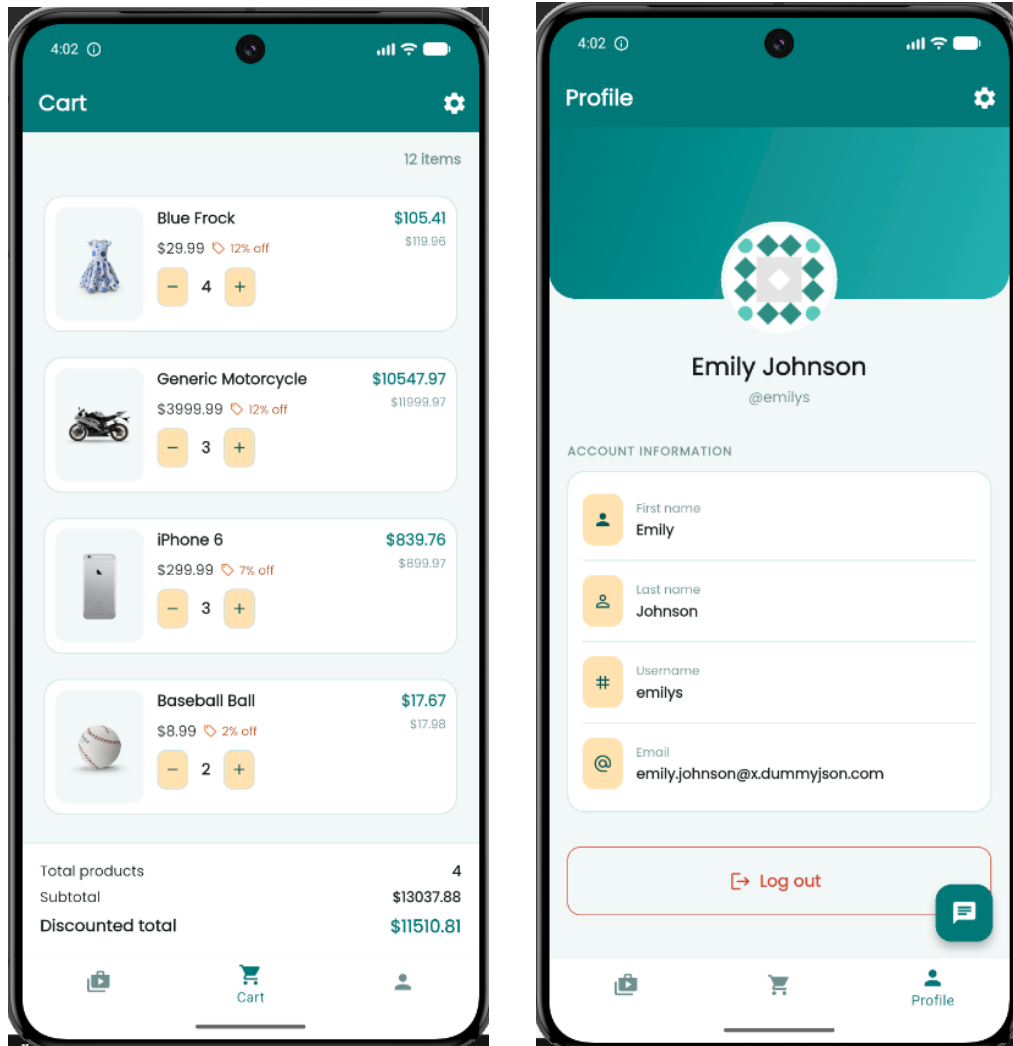
Enhancement 1: Make your own UI for the splash_screen implementing the persistent authentication.



Enhancement 2: Make your own UI for the sign_screen implementing the user_service and the authentication logic above.



Enhancement 3: Using the `user_service` create your own `user.dart` (model) implementing it on this project and rendering the data on the `profile_screen` creating UI on it. Based on the saved user data render the cart by `userId`



Laboratory Discussion: Discuss how the user model, services and screen interact with each other to render the API endpoint going to the `profile_screen`. Discuss the updated design pattern in this activity. Also discuss how to use the saved data in rendering the `cart_screen` by user `id`.

The user feature is divided into three parts, with each part handling a specific responsibility.

- **models/user.dart** holds the user data, with `User.fromJson()` converting the API response into a `User` object and `displayName` combining the first and last name.
- **services/user_service.dart** handles the API and saved user data: `loginUser()` sends the username and password to `POST /auth/login`, then saves the returned user details and token in `SharedPreferences`.
- **screens/profile_screen.dart** does not handle HTTP or JSON. It simply gets the saved user through `getUser()`, stores it in `_user`, and displays the profile information. Since the user data is saved on the phone during login, the profile can load the information quickly and even without an internet connection.

`UserService` also handles login persistence. `isLoggedIn()` checks if a token exists, while `logout()` clears the saved data. This allows the app to skip the sign-in form when a user is already logged in. The saved user ID also fixes the cart issue: instead of using a hardcoded current user from the `DummyJson`. **cart_screen.dart** gets the logged-in user through `getUser()` and passes `user.id` to `getCartsByUserId()`, so each account gets its own cart. The two services do not directly call each other—the screen connects them by getting the user ID from `UserService` and passing it to `CartService`, while `SharedPreferences` stores the user data between screens.

Lab Activity Rubric {*Copy and paste the Rubric here*}

Name	Jamaine Grace Tuazon
Section	INF231
Date	August 21, 2026

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Code Implementation	Code runs flawlessly, includes all required features (placeholders, avatars, widgets for like/comment).	Code mostly correct, minor errors, or missing one minor feature.	Noticeable errors or missing two or more features.	Code does not run or is significantly incomplete.	
Design Consistency	UI closely resembles intended layout, proper spacing, alignment, and styling.	Layout mostly accurate, minor spacing or alignment issues.	Layout deviates significantly from the intended design or appears incomplete.	Layout is unrecognizable or does not follow the instructions.	
Use of Widgets and Enhancements	Properly implements widgets for placeholders, avatars, and like/comment buttons.	Most widgets are correctly implemented, with minor issues or missing one enhancement.	Widgets are inconsistently used or missing two or more enhancements.	Widgets are not used, or enhancements are not implemented.	
Code Organization and Readability	Code is well-structured, follows best practices, and adheres to the specified file organization.	Code is mostly organized and readable, minor deviations from the required structure.	Code is poorly organized or does not follow the specified structure.	Code is disorganized and difficult to understand.	
Back-End Integration	Back-end integration works flawlessly with efficient API calls, secure data handling, and real-time updates; no errors or downtime.	Back-end integration is mostly correct with only minor API, data, or response handling issues.	Back-end integration has noticeable flaws or inefficiencies; multiple API calls fail or data handling is inconsistent.	Back-end integration has major issues; frequent failures, poor response handling, or insecure practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/20) * 100	